CS 591 Lab Notebook

Lab 3

Name: Jarren Calizo
OdinID: calizo
Date: April 27, 2025

Table of Contents

1. *Lab Overview* 2

2. *Lab 3.1 Encoding and hashes*
    - Step 2: Using Base64 and OpenSSL 3
    - Step 3: Cryptographic Hash Functions 5
    - Step 4: Salt & Stretch 6
    - Step 5: Hash Types 7
    - Step 6: Cracking Passwords with Hashcat 8
    - Step 7: Practice — Cracking Hashes 9
    - Step 8: Cracking SSH Passphrases 13

3. *Lab 3.2 Encryption*
    - Step 1: Symmetric Encryption — AES-256-ECB Mode 15
    - Step 2: Symmetric Encryption — AES-256-CBC Mode 16
    - Step 3 & 4: Diffie-Hellman Secret Key Generation 17
    - Step 5: Asymmetric Encryption (RSA Key Setup) 20
    - Step 6: Encryption and Decryption using RSA Public Key 21
    - Step 7 & 8: Combining Symmetric and Asymmetric Encryption 23

4. *Lab 3.3: Digital Signatures*
    - Steps 1–3 Completion (Alice to Bob Authentication) 25
    - Steps 4–7: Certificate Authorities and Trust Chains 26
    - Steps 8–11: Setting Up HTTPS with Let's Encrypt and Digital Certificates 29
    - Step 12 & 13: Certificate Transparency Logs and Cleanup 32

Lab Overview

Lab 3 provided an in-depth, hands-on exploration of core security primitives — encoding, hashing, encryption, key exchange, digital signatures, and web certificate management — by applying them across real-world scenarios.

In Lab 3.1: Encoding and Hashes, I began by verifying how Base64 encoding transforms byte patterns, observing firsthand how minor input changes lead to major output differences, highlighting cryptographic hash properties like the avalanche effect. I then reinforced the importance of salting and stretching passwords, tested the effectiveness of crack tools like Hashcat and John-the-Ripper, and cracked a series of password hashes with real-world wordlists, gaining a practical perspective on modern password security challenges.

In Lab 3.2: Encryption, I implemented symmetric encryption schemes (AES-256-ECB and AES-256-CBC), witnessed the risks of predictable encryption in ECB mode, and saw how IVs enhance CBC mode’s security. I followed this by building a Diffie-Hellman key exchange workflow, confirming both parties derived identical shared keys without direct transmission. Using RSA asymmetric encryption, I also learned the size limitations for encrypting data and designed a hybrid encryption system — encrypting large files with AES and securing the symmetric key via RSA.

Finally, in Lab 3.3: Digital Signatures, I simulated public key authentication between Alice and Bob, issued and verified certificates using a mini Certificate Authority, and secured a live web server with HTTPS through Let’s Encrypt. I manually configured nginx for a custom domain (34.53.44.66.nip.io via nip.io), deployed certbot to obtain a valid certificate, inspected certificate transparency logs through crt.sh, and analyzed both the public and private key structures supporting the HTTPS ecosystem. I wrapped up the lab by cleaning up all cloud resources to avoid unnecessary charges.

Together, Lab 3 significantly deepened my applied understanding of how encoding, encryption, authentication, and trust infrastructure work together to secure communications — from simple bytes to global internet certificates.

Lab 3.1 Encoding and hashes

Step 2: Using Base64 and OpenSSL

Encode 3 null bytes and verify Base64 output

To verify that three null bytes produce four 'A' characters when Base64 encoded, I ran:

- echo -ne '\x0\x0\x0' | openssl enc -base64

and

- python3 -c 'import sys; sys.stdout.buffer.write(3*b"\x00")' | openssl enc -base64

Both commands correctly output AAAA, confirming the expected behavior.

Write null bytes to input.txt and encode with OpenSSL

I generated an input.txt containing three null bytes:

- python3 -c 'import sys; sys.stdout.buffer.write(3*b"\x00")' > input.txt

Then encoded it without a trailing newline:

- openssl enc -base64 -A -in input.txt -out output.txt

Running xxd output.txt confirmed the contents as 4141 4141 (ASCII for AAAA).

Decode output.txt and confirm original bytes

I decoded the Base64 output:

- openssl enc -base64 -A -d -in output.txt -out decode.txt

Viewing decode.txt with xxd showed three null bytes (00 00 00), validating that encoding and decoding preserved the data.

Base64 encode six bytes \x00\x01\x02\x03\x04\x05

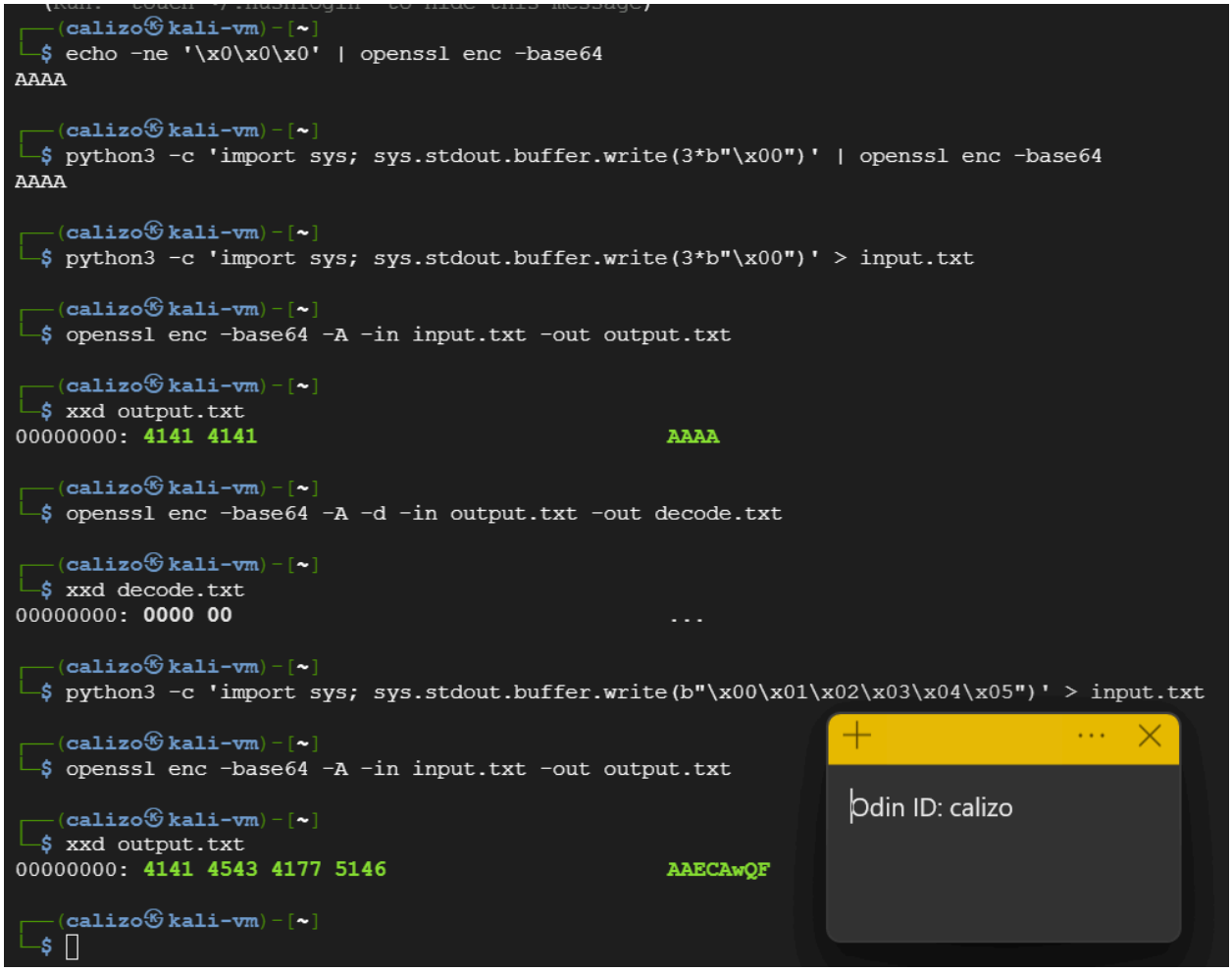
I generated six bytes and wrote them to input.txt:

- python3 -c 'import sys; sys.stdout.buffer.write(b"\x00\x01\x02\x03\x04\x05")' > input.txt

After encoding:

- openssl enc -base64 -A -in input.txt -out output.txt

xxd output.txt confirmed an 8-character Base64 output AAECAwQF, as expected.

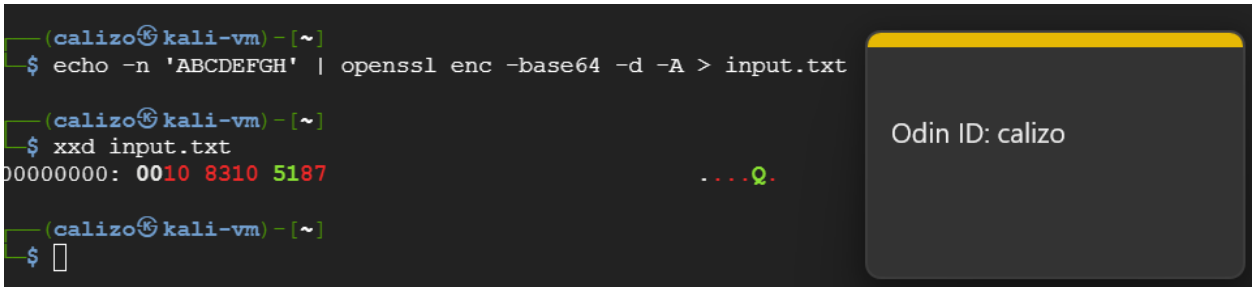


Decode "ABCDEFGH" Base64 string

Finally, I decoded the given string:

- echo -n 'ABCDEFGH' | openssl enc -base64 -d -A > input.txt

xxd input.txt displayed the corresponding six bytes, completing the task.



Step 3: Cryptographic Hash Functions

Generate SHA-256 hashes for 'Password' and 'password'

Using OpenSSL, I computed the SHA-256 hashes for both 'Password' and 'password':

- echo -n 'Password' | openssl dgst -sha256 | awk '{print $2}'
- echo -n 'password' | openssl dgst -sha256 | awk '{print $2}'

Even though the two strings differed by only a single character case, the resulting hashes were completely different, illustrating the strong avalanche effect property of cryptographic hash functions.

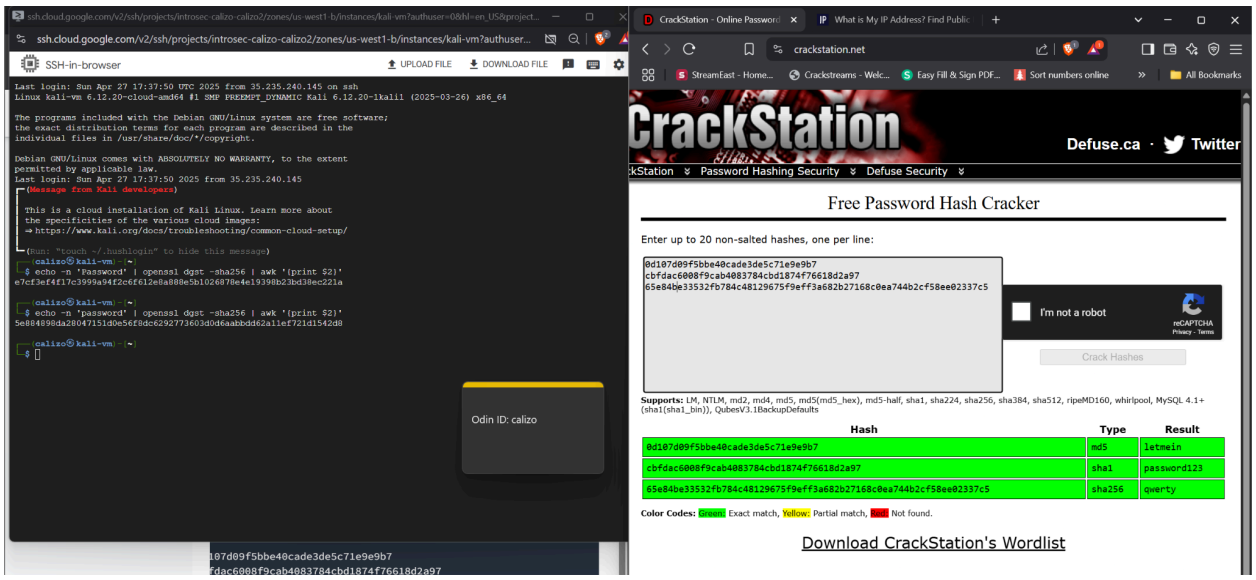
Identify the attacked property

The property under attack here is **pre-image resistance**. Since hash values for common inputs are precomputed and searchable (e.g., rainbow tables), adversaries can reverse a known hash back to its original input, violating the expectation that finding a matching input should be infeasible.

Search provided hashes on CrackStation

I submitted the following hashes to CrackStation to identify their original plaintext values:

- 0d107d09f5bbe40cade3de5c71e9e9b7 → letmein (MD5)
- cbfdac6008f9cab4083784cbd1874f76618d2a97 → password123 (SHA-1)
- 65e84be33532fb784c48129675f9eff3a682b27168c0ea744b2cf58ee02337c5 → qwerty (SHA-256)



Step 4: Salt & Stretch

Generate salted SHA-256 hash and search CrackStation

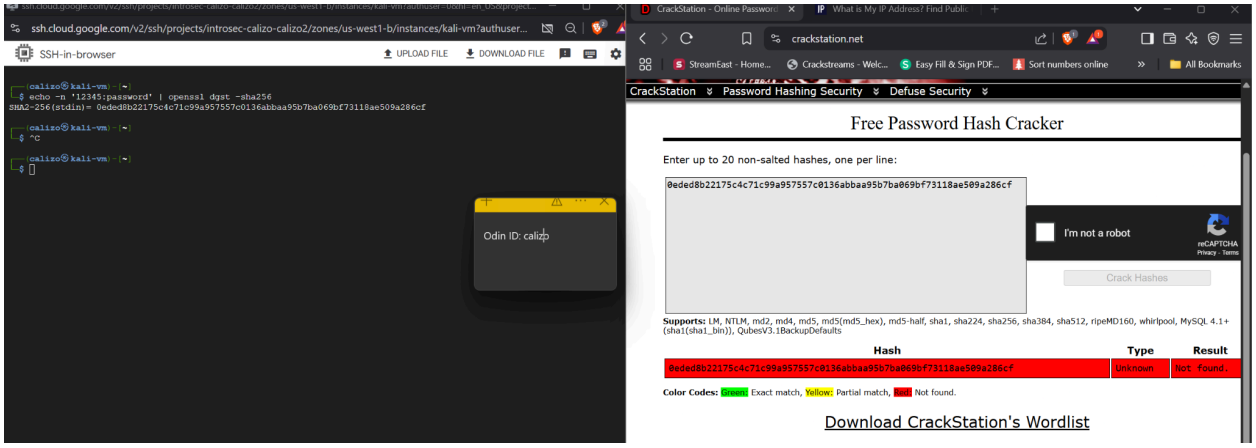
Using OpenSSL, I generated the salted hash of the password:

- echo -n '12345:password' | openssl dgst -sha256

This produced the hash:

- 0eded8b22175c4c71c99a957557c0136abbaa95b7ba069bf73118ae509a286cf

I submitted the hash to CrackStation for lookup.



This screenshot was captured showing that the hash was not found (colored red), demonstrating that even a small salt like "12345" renders the precomputed rainbow tables ineffective.

Measure SHA-256 and PBKDF2 execution times

I measured the time to compute a simple SHA-256 hash and two PBKDF2 hashes at different iteration counts:

Command	real time
SHA-256	0m0.010s
PBKDF2 (31,000 iterations)	0m0.031s
PBKDF2 (310,000 iterations)	0m0.205s

Question 2: How much slower are the PBKDF hashes?

PBKDF2 with 31,000 iterations took approximately 3× longer than simple SHA-256.
PBKDF2 with 310,000 iterations took approximately 20× longer than simple SHA-256.

This confirms that hash stretching significantly slows down brute-force attacks, making password cracking computationally expensive for attackers.

Step 5: Hash Types

Find the hash mode numbers for specific algorithms

Consulting the man hashcat output, I identified the following mode numbers:

Algorithm	Hashcat Mode Number
MD5	0
HMAC-SHA1	150
NTLM (Windows passwords)	1000
SHA-256	1400
SHA-512 (Unix)	1800

Search for the guest account hash in /etc/shadow

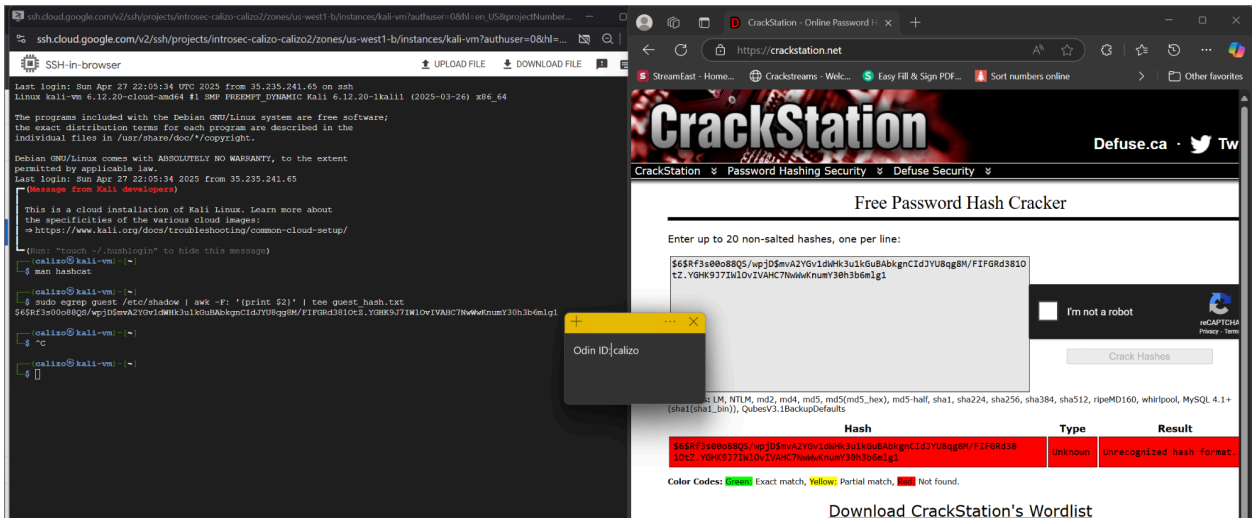
Using the following command:

- sudo egrep guest /etc/shadow | awk -F: '{print $2}' | tee guest_hash.txt

I extracted the guest account's password hash:

- 6Rf3s00o88QS/wpjD$mvA2YGv1dWHk3u1kGuBAbkgnCIdJYU8qg8M/FIFGRd381OtZ.YGHK9J7IWlOvIVAHC7NwWwKnumY30h3b6mlg1

I attempted to search this hash on [CrackStation.net], but as expected, the search failed because of the presence of a random salt and modern hashing scheme, leading to an "Unrecognized hash format" result.



Screenshot captured showing the failed lookup.

Identify the algorithm used (hashid)

Running hashid against the extracted hash:

hashid guest_hash.txt

revealed the hashing algorithm as:

Answer for Question 2:

- The password hash for the guest account was created using **SHA-512 Crypt**.

This format is commonly used for securely storing Linux system passwords and incorporates both salting and stretching mechanisms.

Step 6: Cracking Passwords with Hashcat

Inspect the RockYou wordlist

I used the following command to view the first ten entries of the rockyou.txt file:

- head -10 /usr/share/wordlists/rockyou.txt

This confirmed that the list contains common real-world passwords.

To determine the number of passwords in the list, I ran:
wc -l /usr/share/wordlists/rockyou.txt

Answer for Question 1:

- The rockyou.txt wordlist contains **14,344,392 passwords**.

Crack the guest account password with hashcat

Knowing from Step 5 that the guest account hash uses SHA-512 Crypt (-m 1800 mode in hashcat), I executed:
hashcat -m 1800 ./guest_hash.txt /usr/share/wordlists/rockyou.txt

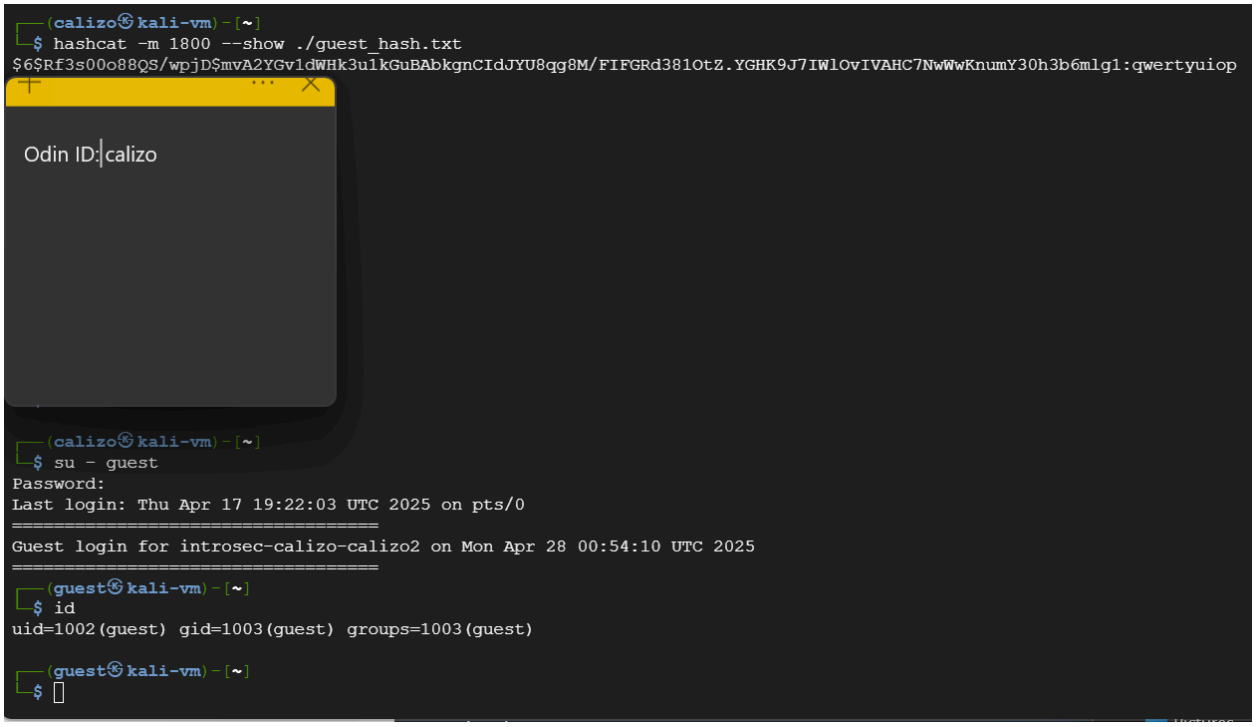
After hashcat successfully cracked the hash, I displayed the cracked password using:
hashcat -m 1800 --show ./guest_hash.txt

The recovered password was:

- qwertyuiop

Validate the cracked password by logging into the guest account

Using the cracked password, I switched to the guest account:
su - guest
Then confirmed the login by running:
id



Screenshot captured showing successful login and guest account details.

Answer for Question 2:

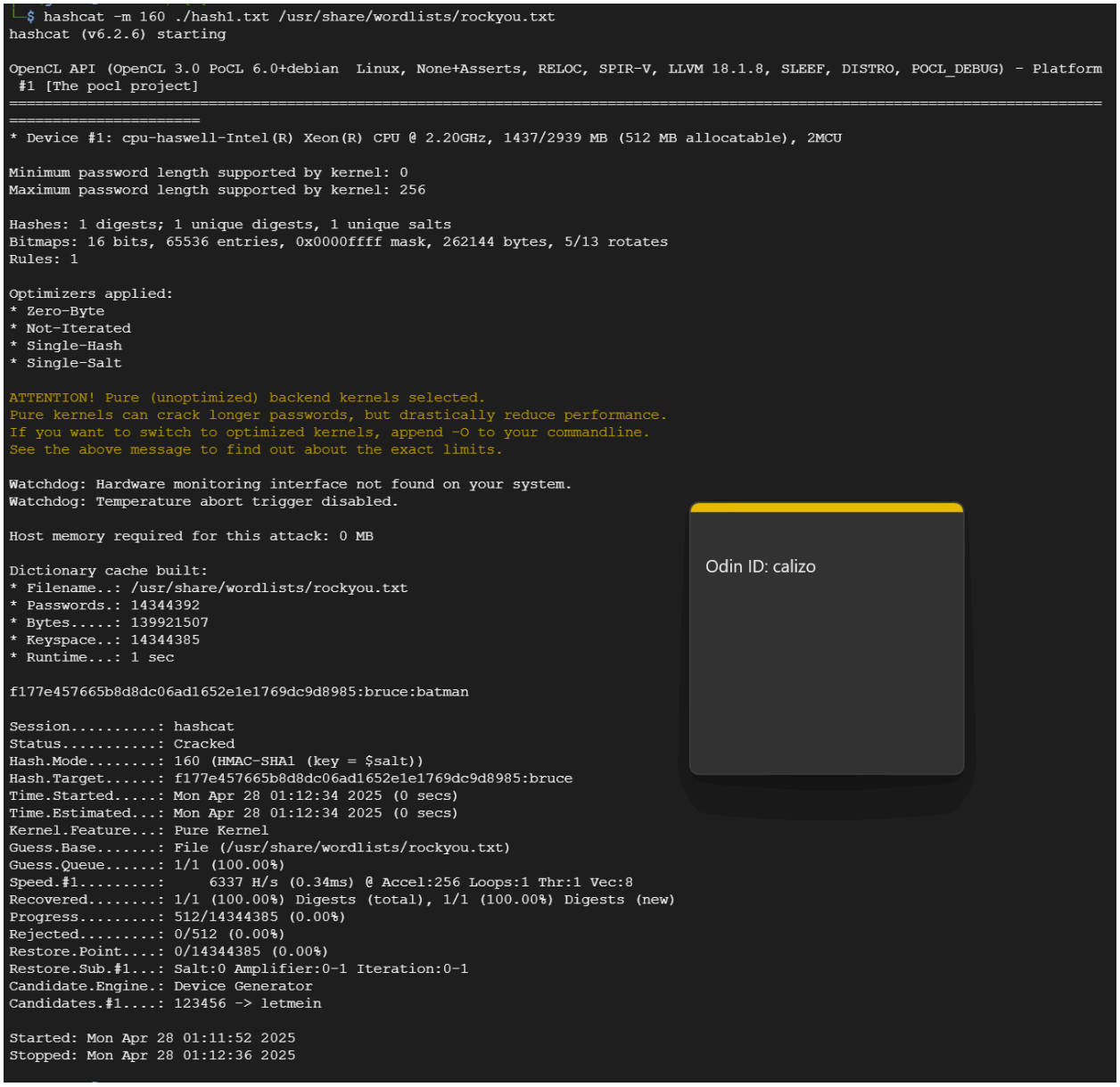
- The cracked password qwertyuiop successfully authenticated me into the guest account.

Step 7: Practice — Cracking Hashes

(1) Crack hash: f177e457665b8d8dc06ad1652e1e1769dc9d8985:bruce
Using hashid, I identified the hash as an HMAC-SHA1 salted hash.
Consulting man hashcat, I selected mode 160 for HMAC-SHA1 (key = salt).

I cracked the hash using:
hashcat -m 160 ./hash1.txt /usr/share/wordlists/rockyou.txt

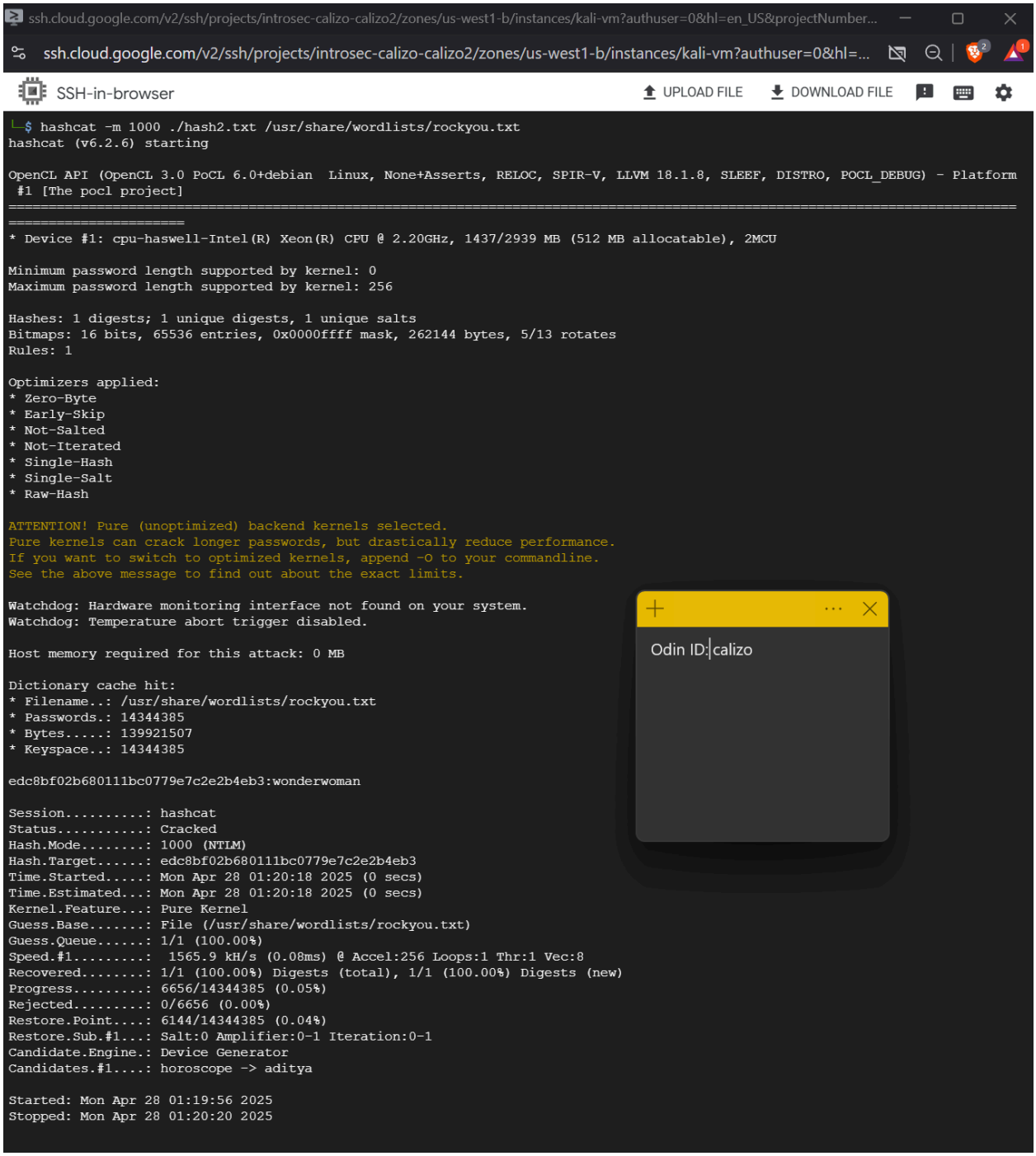
- The recovered password was: batman.



Screenshot captured showing successful crack and matching output.

• (2) Crack hash: edc8bf02b680111bc0779e7c2e2b4eb3
Initially, hashid suggested multiple candidates.
Upon reattempting with mode 1000 (NTLM), the hash was successfully cracked:
hashcat -m 1000 ./hash2.txt /usr/share/wordlists/rockyou.txt

- The recovered password was: wonderwoman.



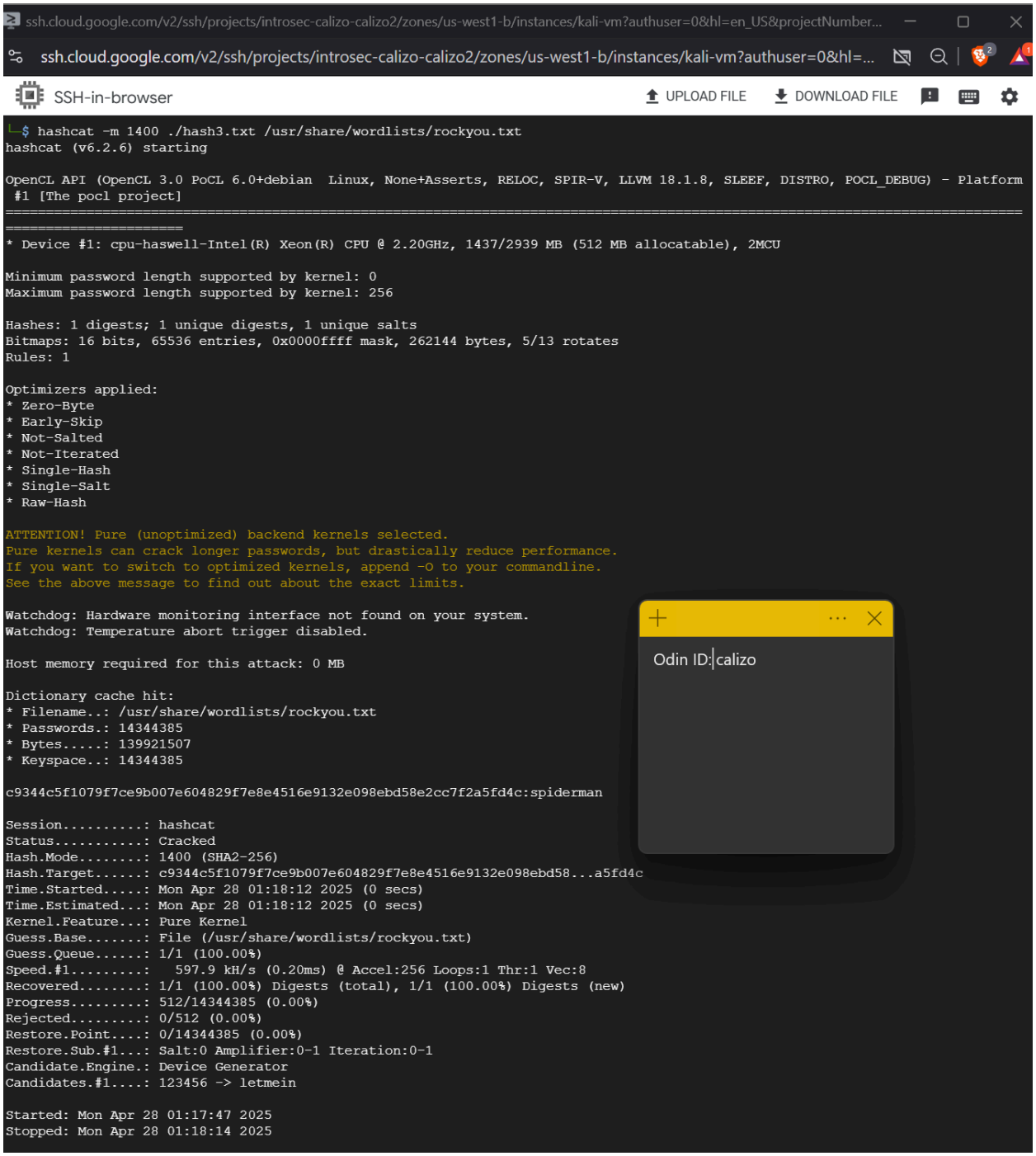
Screenshot captured showing hashcat’s cracked output.

• (3) Crack hash:
c9344c5f1079f7ce9b007e604829f7e8e4516e9132e098ebd58e2cc7f2a5fd4c
Hashid identified this as a SHA-256 hash.

I cracked it using mode 1400:

hashcat -m 1400 ./hash3.txt /usr/share/wordlists/rockyou.txt

- The recovered password was: spiderman.



Screenshot captured with successful match.

• (4) Crack hash: 6xmen$4ElqgJvKoYyAJ4ccK8wTQXvAfw5gvZI3pQCosX9tvf3Tz8q8klYIlfBUOPUVtkmVpigiJsr4ESpmQffW84NvE0

Given the 6 prefix, this indicated a SHA-512 Crypt format.

I cracked it using mode 1800:
hashcat -m 1800 ./hash4.txt /usr/share/wordlists/rockyou.txt

- The recovered password was: phoenix.

```
└─$ hashcat -m 1800 ./hash4.txt /usr/share/wordlists/rockyou.txt
hashcat (v6.2.6) starting

OpenCL API (OpenCL 3.0 PoCL 6.0+debian  Linux, None+Asserts, RELOC, SPIR-V, LLVM 18.1.8, SLEEF, DISTRO, POCL_DEBUG) - Platform
 #1 [The pocl project]
============================================================================================================================
====================
* Device #1: cpu-haswell-Intel(R) Xeon(R) CPU @ 2.20GHz, 1437/2939 MB (512 MB allocatable), 2MCU

Minimum password length supported by kernel: 0
Maximum password length supported by kernel: 256

Hashes: 1 digests; 1 unique digests, 1 unique salts
Bitmaps: 16 bits, 65536 entries, 0x0000ffff mask, 262144 bytes, 5/13 rotates
Rules: 1

Optimizers applied:
* Zero-Byte
* Single-Hash
* Single-Salt
* Uses-64-Bit

ATTENTION! Pure (unoptimized) backend kernels selected.
Pure kernels can crack longer passwords, but drastically reduce performance.
If you want to switch to optimized kernels, append -O to your commandline.
See the above message to find out about the exact limits.

Watchdog: Hardware monitoring interface not found on your system.
Watchdog: Temperature abort trigger disabled.

Host memory required for this attack: 0 MB

Dictionary cache hit:
* Filename..: /usr/share/wordlists/rockyou.txt
* Passwords.: 14344385
* Bytes.....: 139921507
* Keyspace..: 14344385

$6$xmen$4ElqgJvKoYyAJ4ccK8wTQXvAfw5gvZI3pQCosX9tvf3Tz8q8klYIlfBUOPUVtkmVpigiJsr4ESpmQffW84NvE0:phoenix

Session..........: hashcat
Status...........: Cracked
Hash.Mode........: 1800 (sha512crypt $6$, SHA512 (Unix))
Hash.Target......: $6$xmen$4ElqgJvKoYyAJ4ccK8wTQXvAfw5gvZI3pQCosX9tvf3...84NvE0
Time.Started.....: Mon Apr 28 01:27:06 2025 (2 secs)
Time.Estimated...: Mon Apr 28 01:27:08 2025 (0 secs)
Kernel.Feature...: Pure Kernel
Guess.Base.......: File (/usr/share/wordlists/rockyou.txt)
Guess.Queue......: 1/1 (100.00%)
Speed.#1.........:      277 H/s (8.76ms) @ Accel:16 Loops:1024 Thr:1 Vec:4
Recovered........: 1/1 (100.00%) Digests (total), 1/1 (100.00%) Digests (new)
Progress.........: 560/14344385 (0.00%)
Rejected.........: 0/560 (0.00%)
Restore.Point....: 544/14344385 (0.00%)
Restore.Sub.#1...: Salt:0 Amplifier:0-1 Iteration:4096-5000
Candidate.Engine.: Device Generator
Candidates.#1....: wilson -> ganda

Started: Mon Apr 28 01:26:10 2025
Stopped: Mon Apr 28 01:27:10 2025
```



Screenshot captured confirming successful crack.

Step 8: Cracking SSH Passphrases

Switch to guest account and attempt SSH login
I switched to the guest account using:

- su - guest

Attempting to SSH into the keyonly account:

- ssh keyonly@localhost

failed as expected, since the private key was protected by an unknown passphrase.

Convert SSH private key to John-the-Ripper format

Using ssh2john.py, I extracted the hash of the private key:

- python3 /usr/share/john/ssh2john.py ~/.ssh/id_rsa > id_rsa.hash

Crack the SSH passphrase with John

I ran John the Ripper against the hash using the RockYou wordlist:

- john --wordlist=/usr/share/wordlists/rockyou.txt id_rsa.hash

After a few minutes, John successfully cracked the passphrase.

I confirmed the cracked passphrase using:

- john --show id_rsa.hash

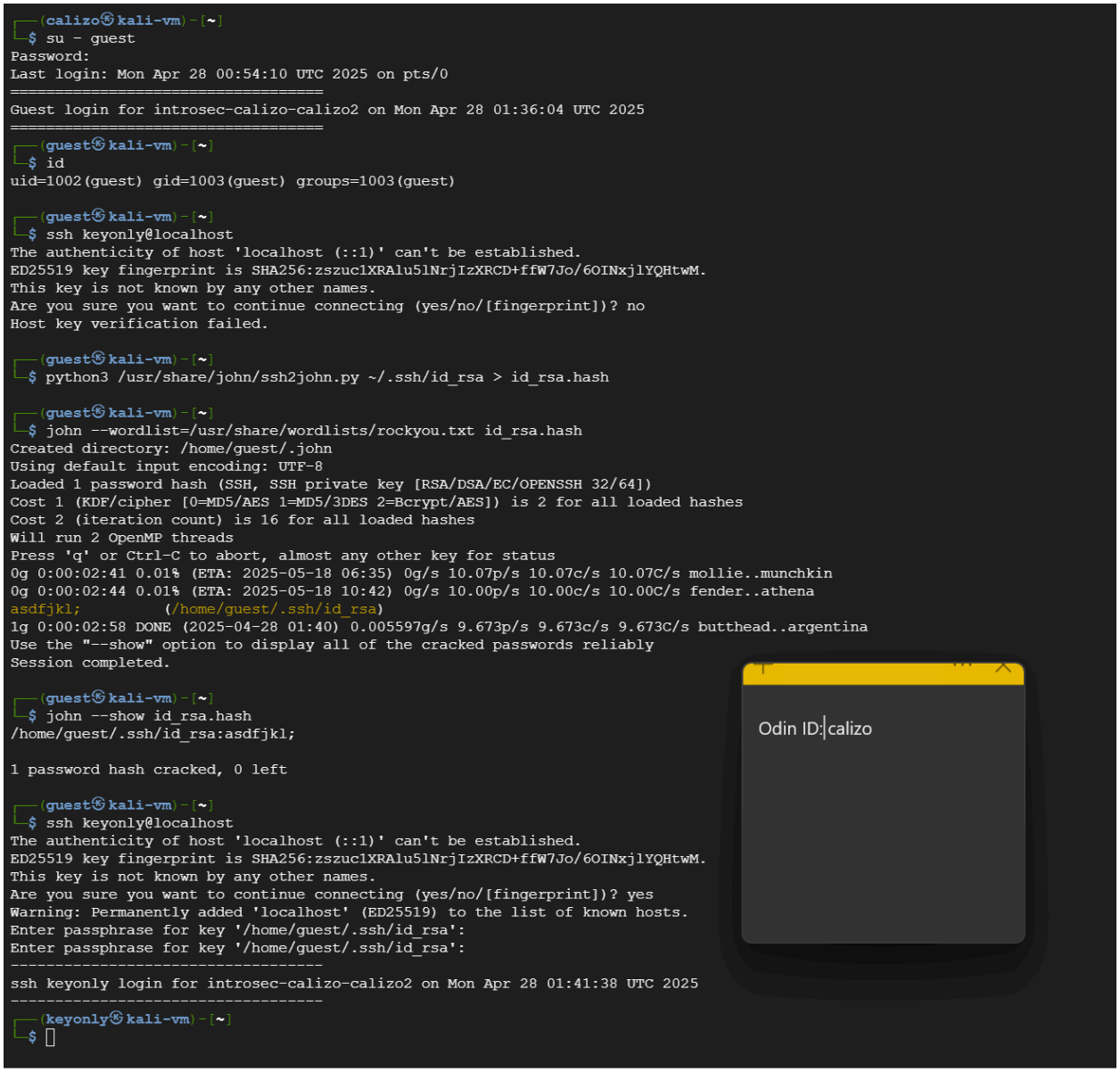
The cracked passphrase was: **asdfjkl;**

Authenticate into keyonly using the cracked passphrase

I retried the SSH login:

- ssh keyonly@localhost

Supplying the cracked passphrase asdfjkl; unlocked the private key, and I successfully logged into the keyonly account.



Screenshot captured showing successful SSH login into keyonly.

Cleanup
Finally, I removed the hash file and exited the session:
rm id_rsa.hash
exit

Lab 3.2 Encryption

Step 1: Symmetric Encryption — AES-256-ECB Mode

Generate a random symmetric key

I generated a random 32-byte symmetric key using:

- openssl rand 32 > sk.bin

This key (sk.bin) was later used for AES encryption operations.

Encrypt 64 "A" characters using AES-256-ECB

I encrypted a string of 64 consecutive "A" characters with AES-256-ECB using the command:

- python3 -c 'print(64*"A")' | openssl enc -aes-256-ecb -pass file:sk.bin - | xxd

The output ciphertext clearly exhibited a repeating pattern.

Question 1: What is the repeated pattern of bytes in the ciphertext?

- The repeated pattern observed in the ciphertext was:
  c5e1 c88c 229c 2dec 764f 153e f724 e6ed

This block was repeated multiple times, which is characteristic of ECB mode's weakness in encrypting identical plaintext blocks.

Question 2: How large is the blocksize in bits?

- AES uses a **128-bit blocksize** (i.e., 16 bytes per block), regardless of the key length (AES-128, AES-192, or AES-256).

The repetition observed confirms that the block size remains at 128 bits.

Step 2: Symmetric Encryption — AES-256-CBC Mode

Encrypt 64 "A"s using AES-256-CBC

I encrypted 64 "A" characters directly through the terminal using:

- python3 -c 'print(64*"A")' | openssl enc -aes-256-cbc -pass file:sk.bin - | xxd

Unlike ECB mode, the resulting ciphertext did not display any repeated patterns, illustrating CBC mode’s resistance to simple pattern leakage.

Symmetric encryption and file-based encryption

I created a plaintext file with 64 "A"s:

- python3 -c 'print(64*"A")' > file.txt

Then encrypted it with AES-256-CBC using:

- openssl enc -aes-256-cbc -in file.txt -out file_sk.bin -pass file:sk.bin
- xxd file_sk.bin

I then repeated the encryption operation:

- openssl enc -aes-256-cbc -in file.txt -out file_sk.bin -pass file:sk.bin
- xxd file_sk.bin

and observed that the ciphertext changed between the two encryptions.

• Question: Why are the encryption outputs different?
- The outputs are different because AES-CBC mode introduces a **randomly generated Initialization Vector (IV)** for every encryption operation. The IV ensures that even identical plaintexts encrypted under the same key produce completely different ciphertexts, enhancing semantic security and preventing pattern analysis.

Decrypt the file and verify
I decrypted the encrypted file back into plaintext:
- openssl enc -d -aes-256-cbc -in file_sk.bin -out decrypt.txt -pass file:sk.bin

and verified the match using:
- diff file.txt decrypt.txt

The diff command returned no output, confirming that the decryption successfully restored the original file.

Issue: How can Alice and Bob securely distribute a symmetric key?
- Alice and Bob can securely distribute the symmetric key by encrypting it using asymmetric encryption (e.g., RSA public key encryption). Alice encrypts the symmetric key with Bob's public key; only Bob's private key can decrypt it, ensuring secure delivery of the shared symmetric key.

Step 3 & 4: Diffie-Hellman Secret Key Generation

Key Setup and Generation
To prepare for Diffie-Hellman key exchange, I first created private subdirectories for Bob and Alice:
- cd
- mkdir Bob Alice

Using OpenSSL, I generated a set of global public parameters (PubDHParams.pem) to define the shared Diffie-Hellman environment:
- openssl genpkey -genparam -algorithm DH -out ~/PubDHParams.pem
- cat ~/PubDHParams.pem
- openssl pkeyparam -in ~/PubDHParams.pem -text -noout

With the parameters established, Bob generated his private key and published his public key:
- cd ~/Bob
- openssl genpkey -paramfile ~/PubDHParams.pem -out BobPrivDHKey.pem
- openssl pkey -in BobPrivDHKey.pem -text -noout

- openssl pkey -in BobPrivDHKey.pem -pubout -out ~/BobPubDHKey.pem
- openssl pkey -pubin -in ~/BobPubDHKey.pem -text -noout

Similarly, Alice generated her private key and exported her public key:
- cd ~/Alice
- openssl genpkey -paramfile ~/PubDHParams.pem -out AlicePrivDHKey.pem
- openssl pkey -in AlicePrivDHKey.pem -pubout -out ~/AlicePubDHKey.pem

Diffie-Hellman Secret Derivation
Once both public keys were available, each participant independently derived the shared secret:

Bob used his private key and Alice’s public key:
- cd ~/Bob
- openssl pkeyutl -derive -inkey BobPrivDHKey.pem -peerkey ~/AlicePubDHKey.pem -out sk.bin

Alice used her private key and Bob’s public key:
- cd ~/Alice
- openssl pkeyutl -derive -inkey AlicePrivDHKey.pem -peerkey ~/BobPubDHKey.pem -out sk.bin

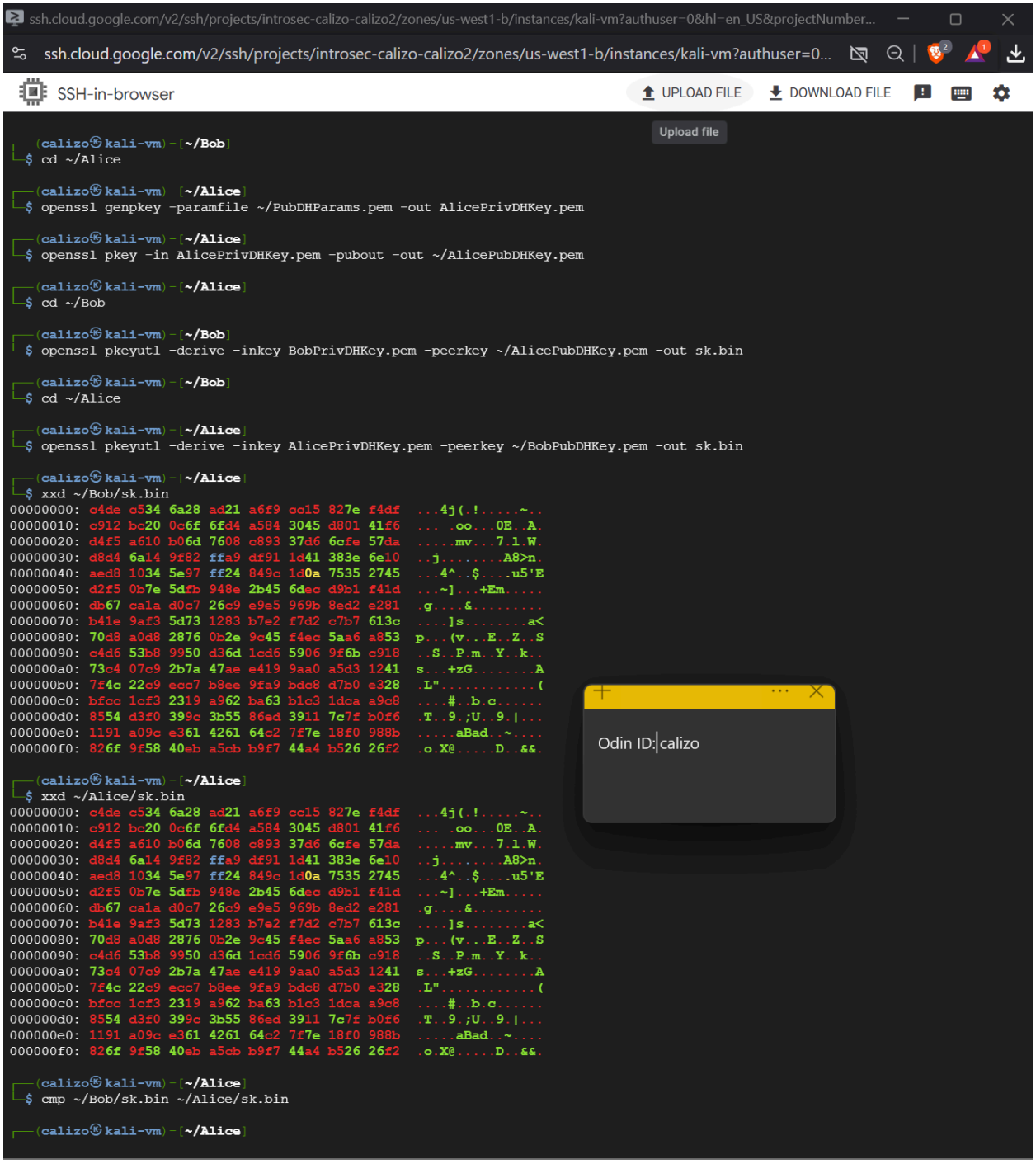
Both outputs were saved as sk.bin within their respective directories.

Key Verification
I examined both Bob and Alice’s derived keys using xxd:
- xxd ~/Bob/sk.bin
- xxd ~/Alice/sk.bin

Observation:
The hexadecimal outputs were identical, confirming that both parties independently computed the same shared secret key.

To further verify, I used cmp:
- cmp ~/Bob/sk.bin ~/Alice/sk.bin

Result:
No differences were reported — Bob and Alice successfully generated a mutual symmetric key without ever transmitting the key itself over the network.



The screenshot clearly shows the identical outputs from xxd for Bob's and Alice's sk.bin files along with my Odin ID sticky visible.

Cleanup

After successful verification, I cleaned up all DH-related files:

- rm ~/PubDHParams.pem ~/BobPubDHKey.pem ~/AlicePubDHKey.pem ~/Alice/sk.bin ~/Bob/sk.bin ~/Alice/AlicePrivDHKey.pem ~/Bob/BobPrivDHKey.pem

This prevents any sensitive material from lingering on the VM.

Step 5: Asymmetric Encryption (RSA Key Setup)

To set up asymmetric encryption for secure communication, I generated RSA keypairs for both Bob and Alice using OpenSSL.

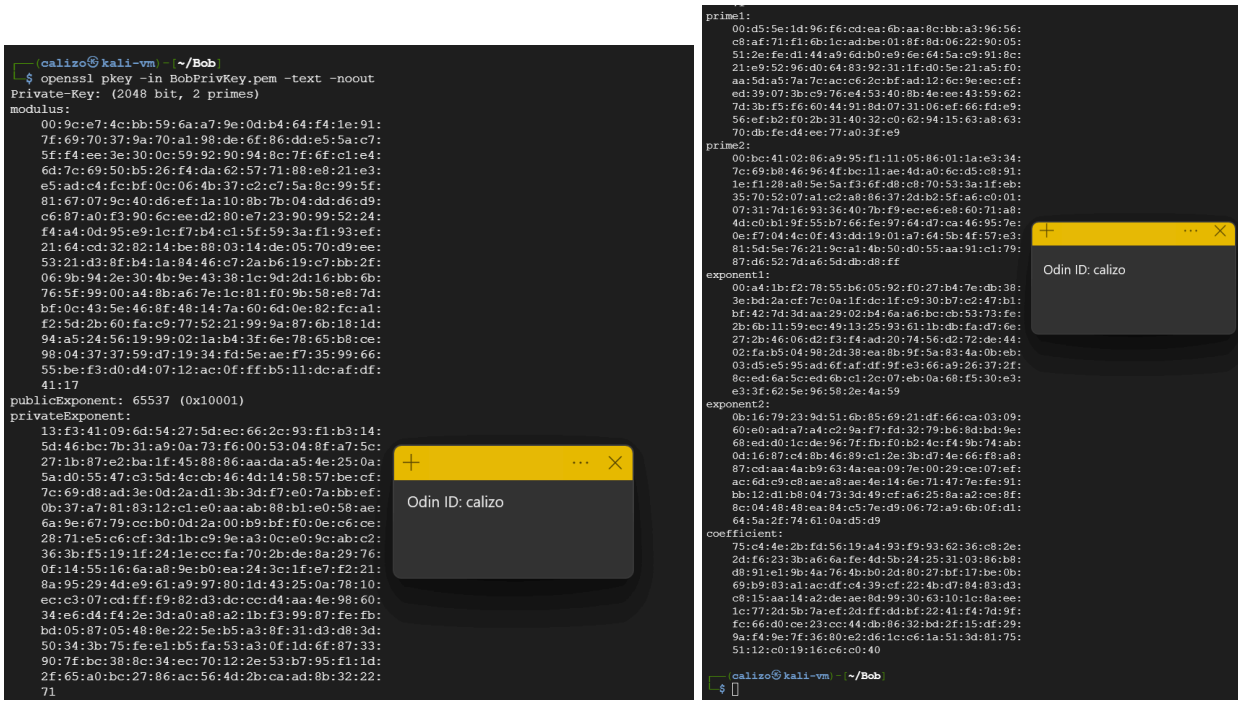
First, I created Bob’s private RSA key using:

- cd ~/Bob
- openssl genpkey -algorithm rsa -out BobPrivKey.pem

Then, I inspected Bob’s private key with:

- openssl pkey -in BobPrivKey.pem -text -noout

Screenshot 1:



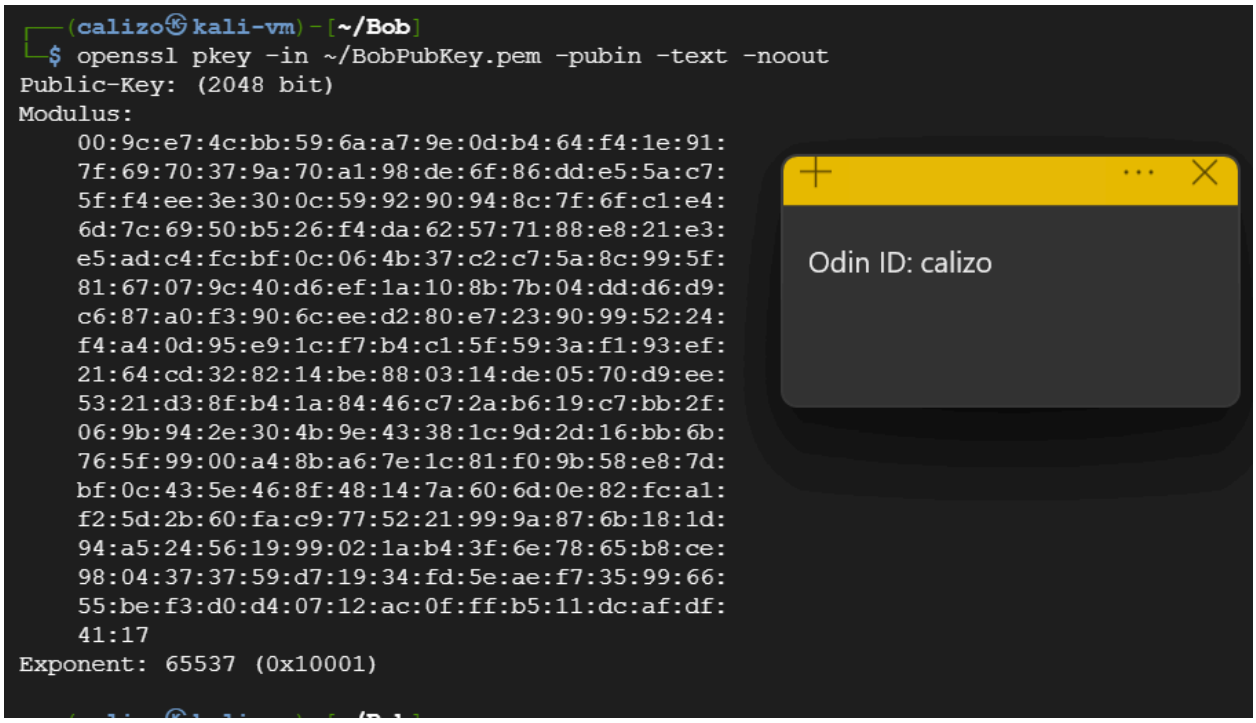
- Displayed the internal RSA structure.
- Captured the two prime numbers (prime1 and prime2) used in generating Bob’s modulus.
- Verified the key strength: **2048 bits**.

Afterward, I extracted Bob’s public key:

- openssl pkey -in BobPrivKey.pem -pubout -out ~/BobPubKey.pem

and examined it:

- openssl pkey -in ~/BobPubKey.pem -pubin -text -noout



Screenshot 2:

- Highlighted the modulus of Bob’s public key.
- Confirmed again that the modulus was **2048 bits**.

Following Bob's key generation, I repeated the process for Alice:

- cd ~/Alice
- openssl genpkey -algorithm rsa -out AlicePrivKey.pem
- openssl pkey -in AlicePrivKey.pem -pubout -out ~/AlicePubKey.pem

This completed the RSA key setup for both parties, preparing them for the next encryption operations.

Step 6: Encryption and Decryption using RSA Public Key

To simulate a real-world use case where Alice encrypts a file directly using Bob’s RSA public key, I first generated a text file containing 512 "A" characters.
However, when attempting encryption with:

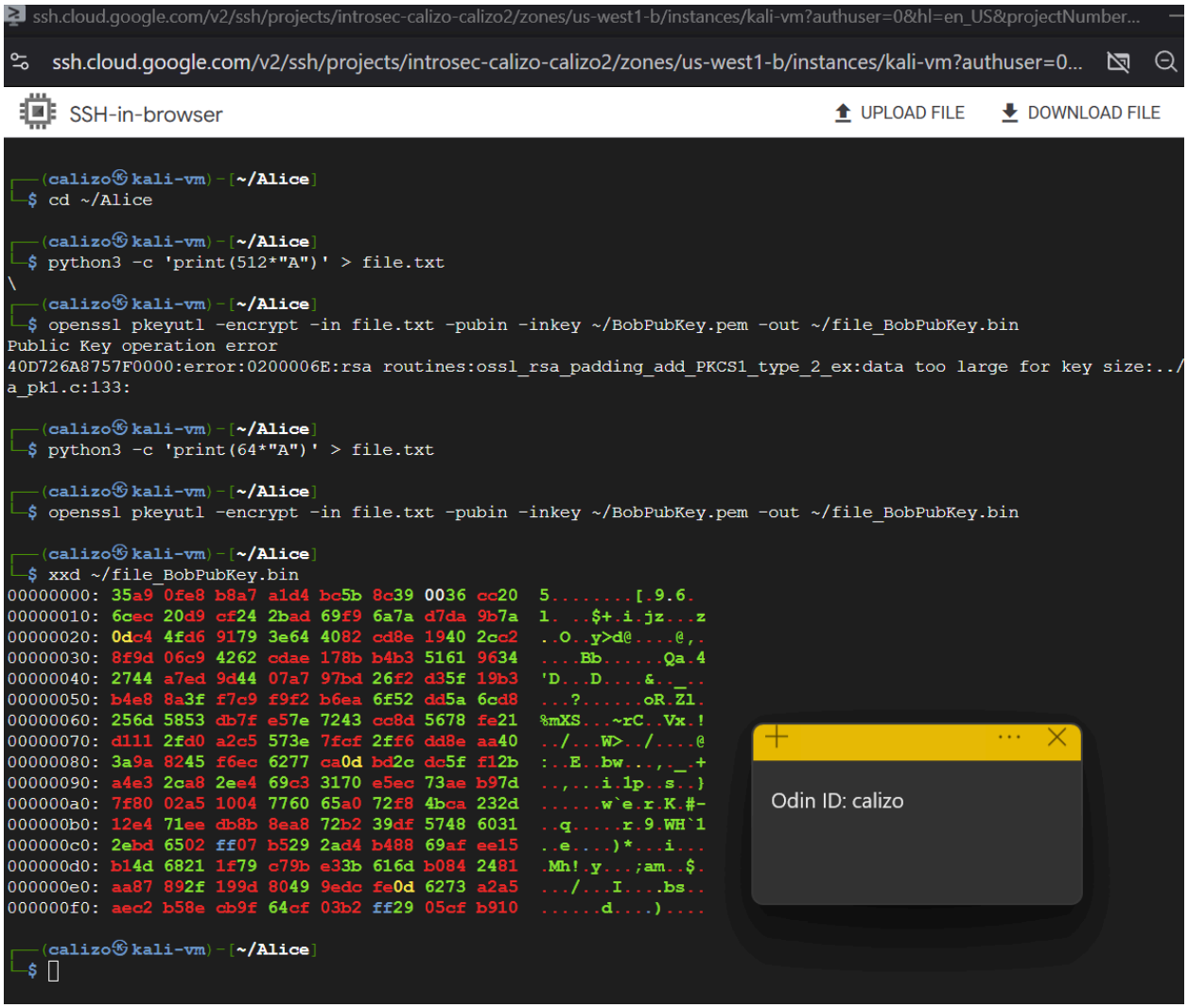
- openssl pkeyutl -encrypt -in file.txt -pubin -inkey ~/BobPubKey.pem -out ~/file_BobPubKey.bin

The encryption failed. The error (data too large for key size) indicated that the input data was too large to be encrypted with RSA directly. This is expected because RSA encryption can only handle messages smaller than the key size (2048 bits ≈ 256 bytes minus padding).

To address this, I regenerated the file with only 64 "A" characters:

- python3 -c 'print(64*"A")' > file.txt
- openssl pkeyutl -encrypt -in file.txt -pubin -inkey ~/BobPubKey.pem -out ~/file_BobPubKey.bin

The encryption succeeded. I verified the ciphertext contents using:
xxd ~/file_BobPubKey.bin
showing successful ciphertext output.



To decrypt, I switched to Bob's directory and used his private key:

- cd ~/Bob
- openssl pkeyutl -decrypt -in ~/file_BobPubKey.bin -inkey BobPrivKey.pem -out decrypt.txt
- diff decrypt.txt ~/Alice/file.txt

The diff command showed no differences, confirming the decrypted text matched the original file exactly.

Question: Why did encrypting 512 bytes fail?

- RSA encryption has a strict size limit based on the key length. A 2048-bit RSA key (256 bytes) cannot encrypt data larger than its modulus size minus padding overhead (for PKCS#1 v1.5 padding, typically around 245 bytes maximum). Attempting to encrypt 512 bytes exceeded this limit, resulting in an expected encryption error.

Issue: How can we send large amounts of data using asymmetric encryption?

- To send large amounts of data securely, we use a hybrid encryption approach:
  - Generate a random symmetric key (e.g., AES-256) to encrypt the large data.
  - Encrypt the small AES key using RSA.
  - Send both the RSA-encrypted AES key and the AES-encrypted data.

This combines RSA's security for key exchange with AES's efficiency for bulk data encryption.

Step 7 & 8: Combining Symmetric and Asymmetric Encryption

In this exercise, I implemented a hybrid encryption scheme to securely send a large file from Alice to Bob. Since asymmetric encryption (RSA) is impractical for encrypting large datasets, I first had Alice encrypt the actual file with a symmetric AES-256 key and only encrypt the AES key itself with RSA, combining the strengths of both techniques.

First, under Alice's account, I generated a 512-byte file (file.txt) filled with "A" characters:

- python3 -c 'print(512*"A")' > file.txt

Then, I created a random 32-byte symmetric key:

- openssl rand 32 > sk.bin

Using this symmetric key, Alice encrypted file.txt into file_sk.bin with AES-256-CBC mode:

- openssl enc -aes-256-cbc -in file.txt -out ~/file_sk.bin -pass file:sk.bin

After encrypting the large data file, Alice protected the symmetric key itself by encrypting it with Bob’s public RSA key, outputting it as sk_BobPubKey.bin:

- openssl pkeyutl -encrypt -in sk.bin -pubin -inkey ~/BobPubKey.pem -out ~/sk_BobPubKey.bin

At this point, Alice had successfully distributed an encrypted symmetric key and an encrypted file without having to manually share the original key — mitigating the symmetric key distribution problem.

Now switching over to Bob’s account, Bob began the decryption process by first using his private key to recover the original symmetric key:

- openssl pkeyutl -decrypt -in ~/sk_BobPubKey.bin -inkey BobPrivKey.pem -out sk.bin

With the recovered AES key, Bob then decrypted the file:

- openssl enc -d -aes-256-cbc -in ~/file_sk.bin -out decrypt.txt -pass file:sk.bin

Finally, I verified that the decrypted file matched Alice’s original by running:

- diff decrypt.txt ~/Alice/file.txt

The diff command returned no differences, confirming successful decryption and recovery of the original file.

I then cleaned up all intermediate and final files:

- rm decrypt.txt sk.bin ~/file_sk.bin ~/sk_BobPubKey.bin ~/Alice/file.txt ~/Alice/sk.bin

Questions:

How can we send large amounts of data using asymmetric encryption?

- We combine symmetric and asymmetric encryption: encrypt the large data using a symmetric cipher (e.g., AES) and only encrypt the much smaller symmetric key with the recipient’s public RSA key.

How does Bob know Alice sent this file?

- Unfortunately, Bob has no proof that the file came from Alice. Since anyone with Bob’s public key could encrypt the file, the sender's authenticity cannot be guaranteed without an additional mechanism like a digital signature. To

verify sender authenticity, Alice would need to sign the encrypted symmetric key or file using her private RSA key.

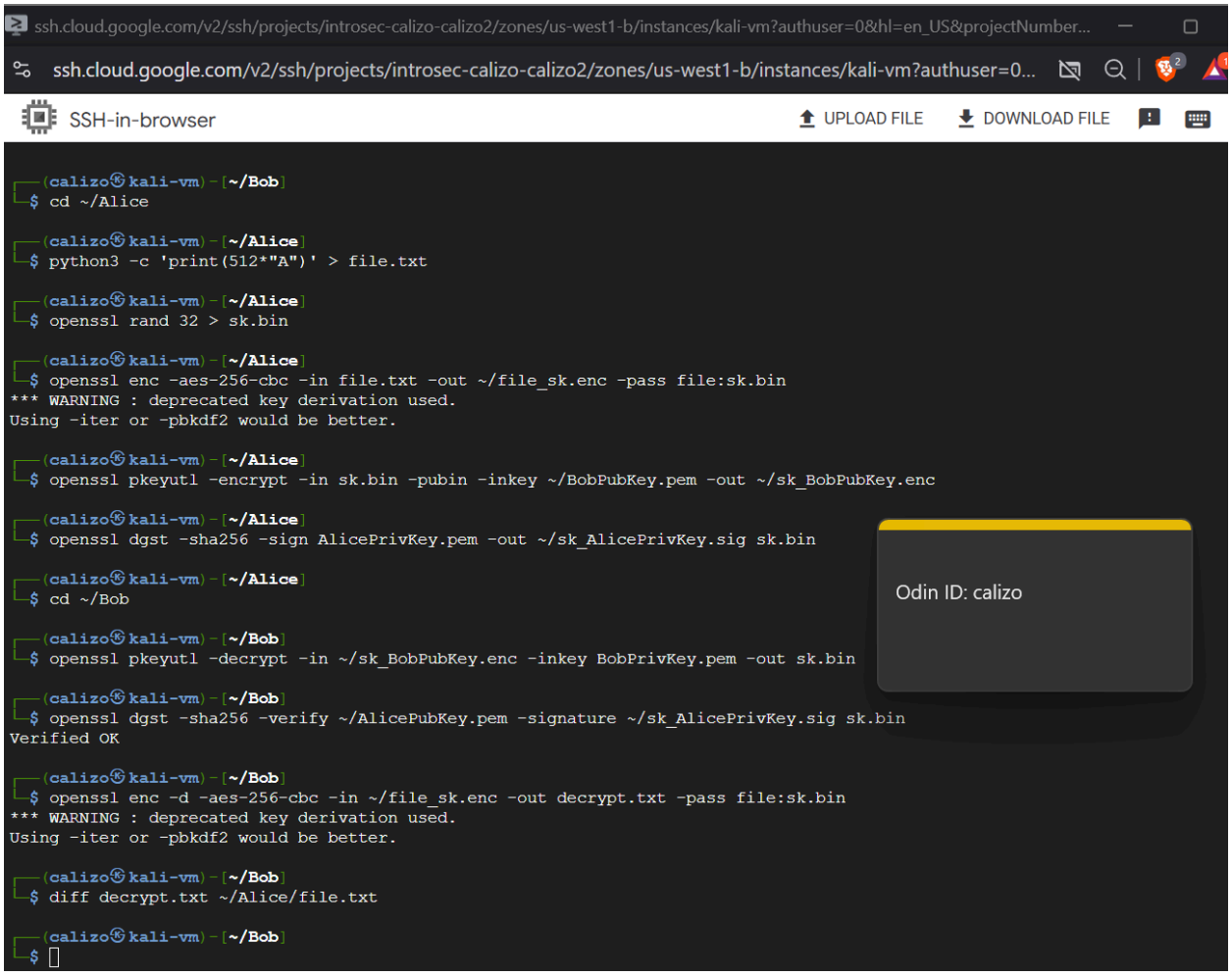
Lab 3.3: Digital Signatures

Steps 1-3 Completion (Alice to Bob Authentication)

To demonstrate how digital signatures authenticate the sender in asymmetric cryptography, I first replicated the setup from the previous lab with an additional signing layer.

Starting from Alice’s directory, I generated a file consisting of 512 "A" characters (file.txt) and created a random 32-byte symmetric key (sk.bin). I encrypted file.txt using AES-256-CBC with the symmetric key, resulting in file_sk.enc. To securely transfer the symmetric key, I encrypted sk.bin using Bob’s public key to create sk_BobPubKey.enc. Finally, to prove the ownership and authenticity of the symmetric key, I computed a SHA-256 hash of sk.bin and digitally signed the hash with Alice’s private key, outputting the signature to sk_AlicePrivKey.sig.

Once the payloads were ready, I switched to Bob’s directory to handle decryption and verification. Bob first decrypted the symmetric key by using his private key on sk_BobPubKey.enc. After recovering sk.bin, he then verified the authenticity of the symmetric key by validating Alice’s digital signature (sk_AlicePrivKey.sig) against the decrypted key. As shown in the terminal, the output confirmed Verified OK, meaning the signature check succeeded, assuring Bob that the symmetric key genuinely came from Alice.



After verification, Bob used the recovered symmetric key to decrypt file_sk.enc, restoring the original file content. A final diff check between Bob’s decrypt.txt and Alice’s original file.txt produced no output, confirming a successful and accurate round-trip of encryption, signing, verification, and decryption.

Through this exercise, I reinforced how digital signatures bind the sender’s identity to the message, enabling not just confidentiality but sender authentication in secure communications.

Steps 4-7: Certificate Authorities and Trust Chains

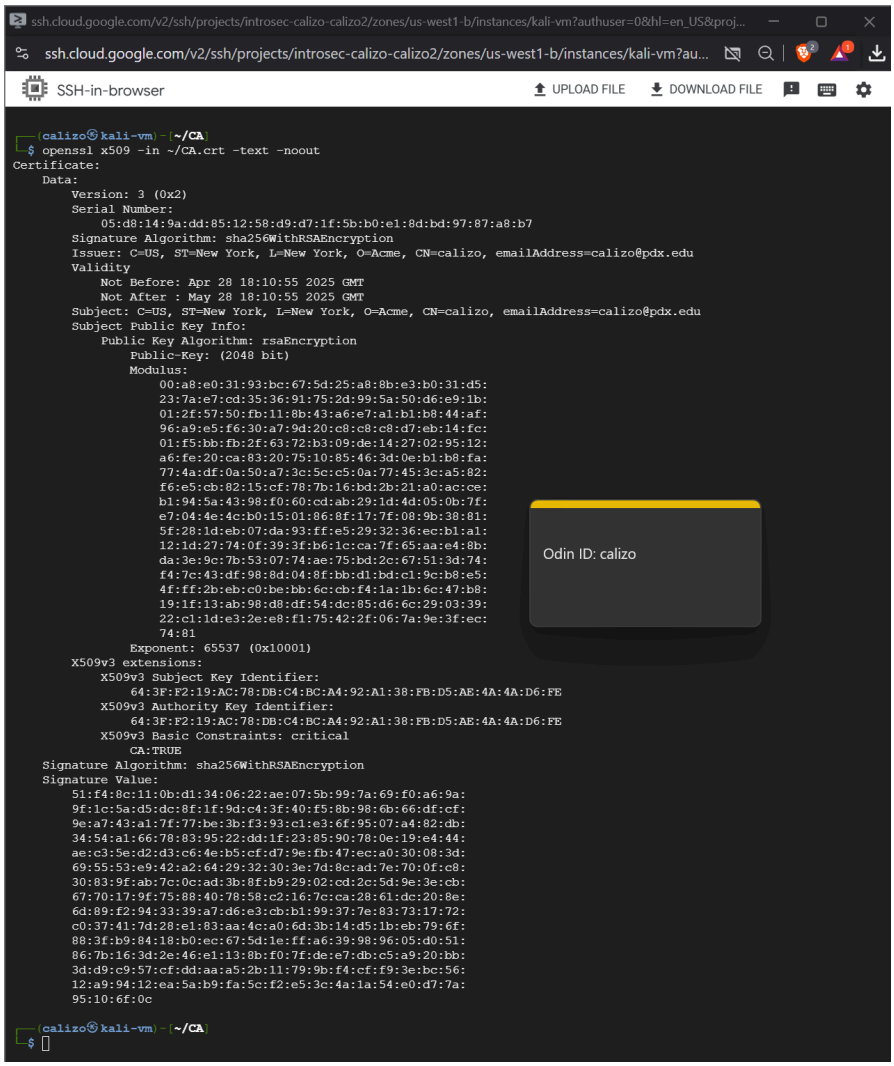
In this portion of the lab, I modeled a simplified Certificate Authority (CA) infrastructure to demonstrate how public keys can be trusted over an insecure network like the Internet.

Step 4: Setting up the CA To start, I created a new directory ~/CA, generated the CA’s private key (CAPrivKey.pem), and issued a self-signed X.509 certificate (CA.crt). During the certificate generation prompts, I populated the Distinguished Name (DN) fields as follows:

- Country: US
- State/Province: New York
- Locality: New York
- Organization: Acme
- Common Name: calizo
- Email: calizo@pdx.edu

Upon inspecting the generated certificate, I verified that the Issuer and Subject fields were identical (C=US, ST=New York, L=New York, O=Acme, CN=calizo, emailAddress=calizo@pdx.edu), confirming the certificate was indeed self-signed.

Answer 1:



Answer 2: The CA certificate is valid for 1 year (until April 28, 2026).

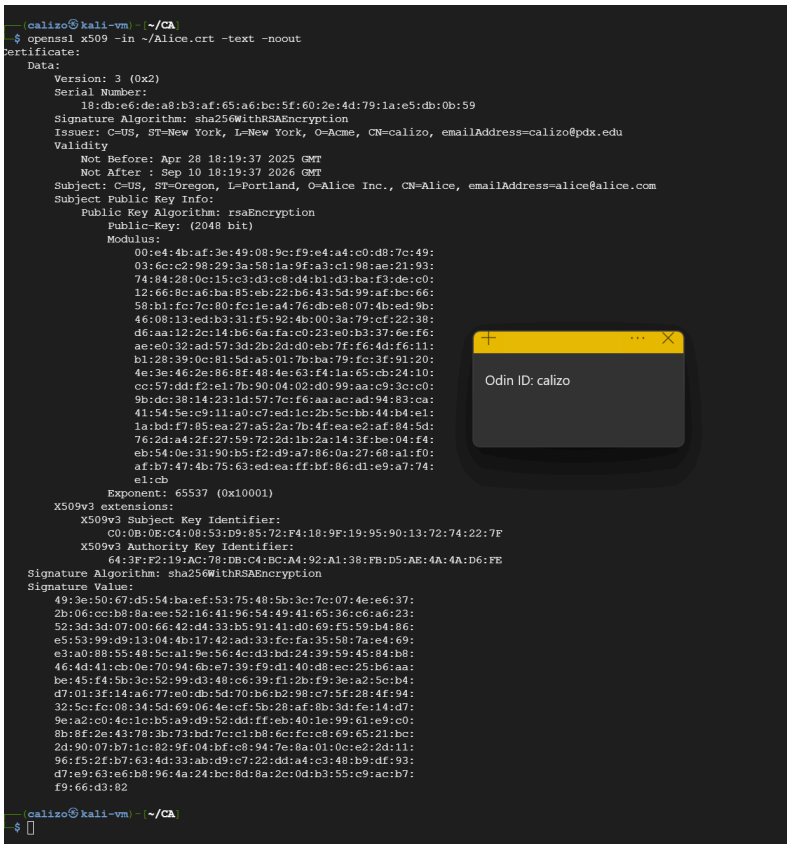
Step 5: Alice’s Certificate Signing Request (CSR) Next, I switched to the ~/Alice directory to prepare Alice’s certificate request. While generating her CSR (AliceSigningReq.csr), I filled in the fields as follows:

- Country: US
- State/Province: Oregon
- Locality: Portland
- Organization: Alice Inc.
- Common Name: Alice
- Email: alice@alice.com

I verified the CSR contents, confirming it included Alice’s public key and identity information, and then "sent" it to the CA directory for signing.

Step 6: Issuing Alice’s Certificate Back in the ~/CA directory, I used the CA’s private key to sign Alice’s certificate request, creating Alice.crt, which binds Alice’s public key to her verified identity. The issuer field was listed as Acme (the CA entity), and the subject matched Alice’s provided fields.

Answer 3:



- Screenshot shows Acme as the Issuer of Alice’s certificate.

Answer 4: Alice’s certificate is valid for 500 days (until September 19, 2026).

Signature Comparison:

- Answer 5: The signature in Alice’s certificate is different from the signature in Alice’s certificate request.
- Answer 6: This difference occurs because the CSR’s signature is generated by Alice’s private key over her public key and metadata, whereas the issued certificate’s signature is generated by the CA’s private key, signing Alice’s public key and validated identity. Thus, they serve different cryptographic roles.

Step 7: Verifying Alice’s Certificate Finally, Bob obtained a copy of Alice’s signed certificate (Alice.crt) and verified it against the CA’s certificate (CA.crt). Running:

- openssl verify -CAfile ~/CA.crt ~/Alice.crt

returned an "OK" output, indicating successful trust validation of Alice’s public key through the CA’s signature.

- Answer 7:



Screenshot shows successful verification output.

By completing these steps, I was able to simulate the real-world flow of public key infrastructure (PKI): a trusted third party (the CA) vouches for Alice’s identity, enabling Bob to trust and securely communicate with her. This structure elegantly solves the key distribution and authentication problem at Internet scale.

Steps 8-11: Setting Up HTTPS with Let's Encrypt and Digital Certificates

In this portion of the lab, I transitioned from serving basic HTTP content on my VM to securing it with HTTPS using Let's Encrypt and NGINX, while following standard digital certificate practices.

Step 8: Setting Up the Web Server

I first created a VM instance using Google Cloud Compute Engine with Ubuntu 22.04 LTS, ensuring HTTP and HTTPS traffic were allowed via the appropriate firewall rules. After installing NGINX (sudo apt install nginx) and verifying my VM’s external IP, I successfully confirmed the unencrypted HTTP server was live by accessing:

- http://34.53.44.66.nip.io

where the NGINX default page loaded properly.

Step 9: Registering a DNS Name

Next, I needed a DNS name pointing to my VM's external IP to satisfy Let's Encrypt requirements. I chose the nip.io wildcard DNS service, where I mapped:

- 34.53.44.66.nip.io

This automatic mapping allowed immediate DNS resolution without additional setup or registration.

Step 10: Configuring NGINX for My Site

After the DNS mapping, I configured a custom NGINX server block. I created the directory and sample page:

- sudo mkdir -p /var/www/34.53.44.66.nip.io
- sudo bash -c 'echo "<html><body><h1>Hello</h1></body></html>" > /var/www/34.53.44.66.nip.io/index.html'

Then, I defined my NGINX site configuration under /etc/nginx/sites-available/34.53.44.66.nip.io with the following block:

```
server {
   listen 80;
   server_tokens off;
   server_name 34.53.44.66.nip.io;
   root /var/www/34.53.44.66.nip.io;
}
```

I enabled it with:

- sudo ln -s /etc/nginx/sites-available/34.53.44.66.nip.io /etc/nginx/sites-enabled/34.53.44.66.nip.io
- sudo systemctl restart nginx

After completing the above, I was able to view my "Hello" page through my DNS URL over HTTP, confirming NGINX was properly set up for my specific domain.

Step 11: Obtaining and Installing a Let's Encrypt Certificate

I proceeded to secure my site with HTTPS by running:

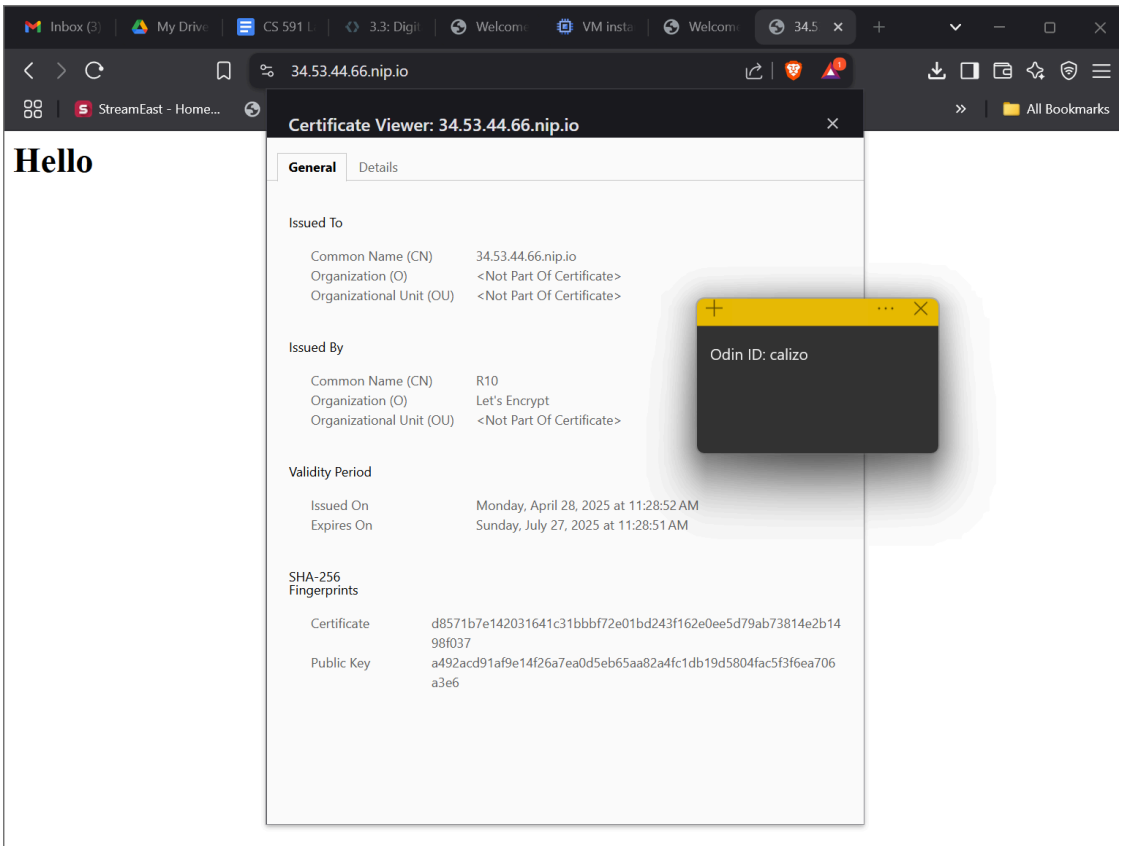
- sudo certbot --nginx

During the certbot prompts:

- I entered my PSU email address for certificate renewal notifications.
- I agreed to the Terms of Service.
- I chose my site 34.53.44.66.nip.io to activate HTTPS for

Upon successful configuration, I visited:
https://34.53.44.66.nip.io
and verified that my browser displayed a padlock icon, indicating a trusted HTTPS connection.

Certificate Inspection and Required Answers:
Upon clicking the padlock and inspecting the certificate, the following details were observed:
(Q1) Screenshot of Certificate:



Screenshot shown above with Certificate Viewer open and "Hello" page

(Q2) How long is the certificate valid for?

- The certificate is valid for **90 days** — specifically, from April 28, 2025, to July 27, 2025.

(Q3) What is the two-letter Common Name (CN) of the issuer?

- The issuer’s Common Name (CN) is **R10**, and the issuing organization is **Let's Encrypt**. The two-letter country code associated with Let's Encrypt (not shown directly here but known) is US.

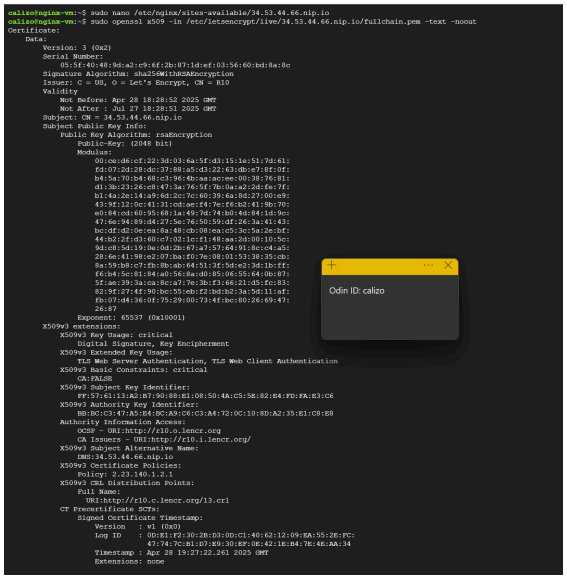
At this stage, my domain 34.53.44.66.nip.io was fully serving content securely over HTTPS, with a trusted Let's Encrypt certificate publicly verifiable through any browser.

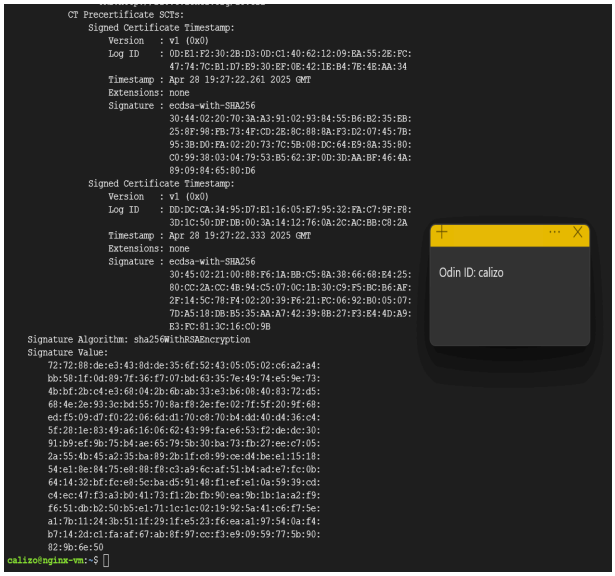
This end-to-end exercise demonstrated how digital signatures, certificate authorities, and web server configuration interact to establish authentic and encrypted communication over the Internet.

Step 12 & 13: Certificate Transparency Logs and Cleanup

Step 12: Examine the certificate
After securing the HTTPS setup in the previous steps, I moved on to examining the certificate information more deeply and finalizing the lab environment.

First, I revisited the nginx configuration file and reviewed the fullchain certificate that Certbot generated (/etc/letsencrypt/live/34.53.44.66.nip.io/fullchain.pem). Using openssl, I extracted the certificate details. I confirmed that the subject public key was a 2048-bit RSA key, consistent with the key size selected during Certbot's issuance process. Additionally, I found the Authority Information Access (AIA) URIs pointing to the Certificate Authority (CA) issuer — in this case, Let's Encrypt via R10 — as well as the Certificate Transparency (CT) pre-certificates embedded in the file.





Screenshot evidence was taken showing:

- The public key for the web server.
- The URLs and URIs for the CA and revocation.
- The SCT (Signed Certificate Timestamps) showing the pre-certificates.

Answer to Question 1:
The public key, AIA URIs, and CT logs were found directly inside the **fullchain.pem**, and screenshots were taken.

Answer to Question 2:
The signature algorithm used to sign the pre-certificates was **ecdsa-with-SHA256** (seen in the CT pre-certificate timestamps section).

Next, I went to crt.sh and searched for my site (34.53.44.66.nip.io). Two certificate IDs were listed:

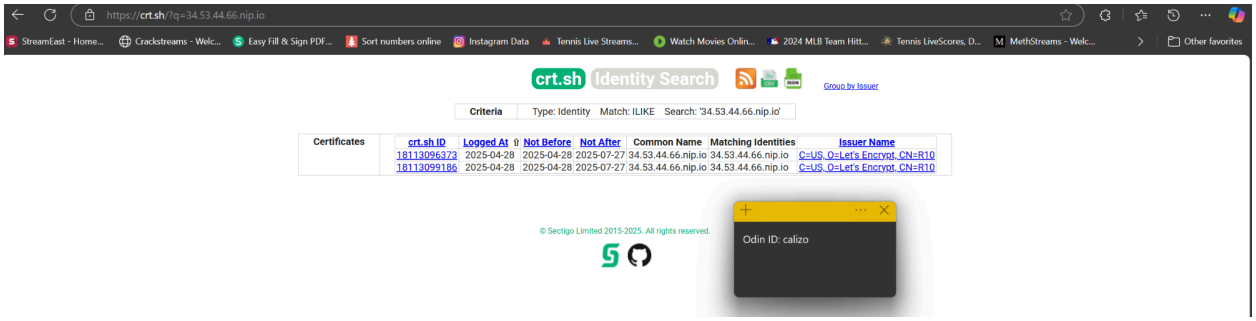
- 18113096373
- 18113099186

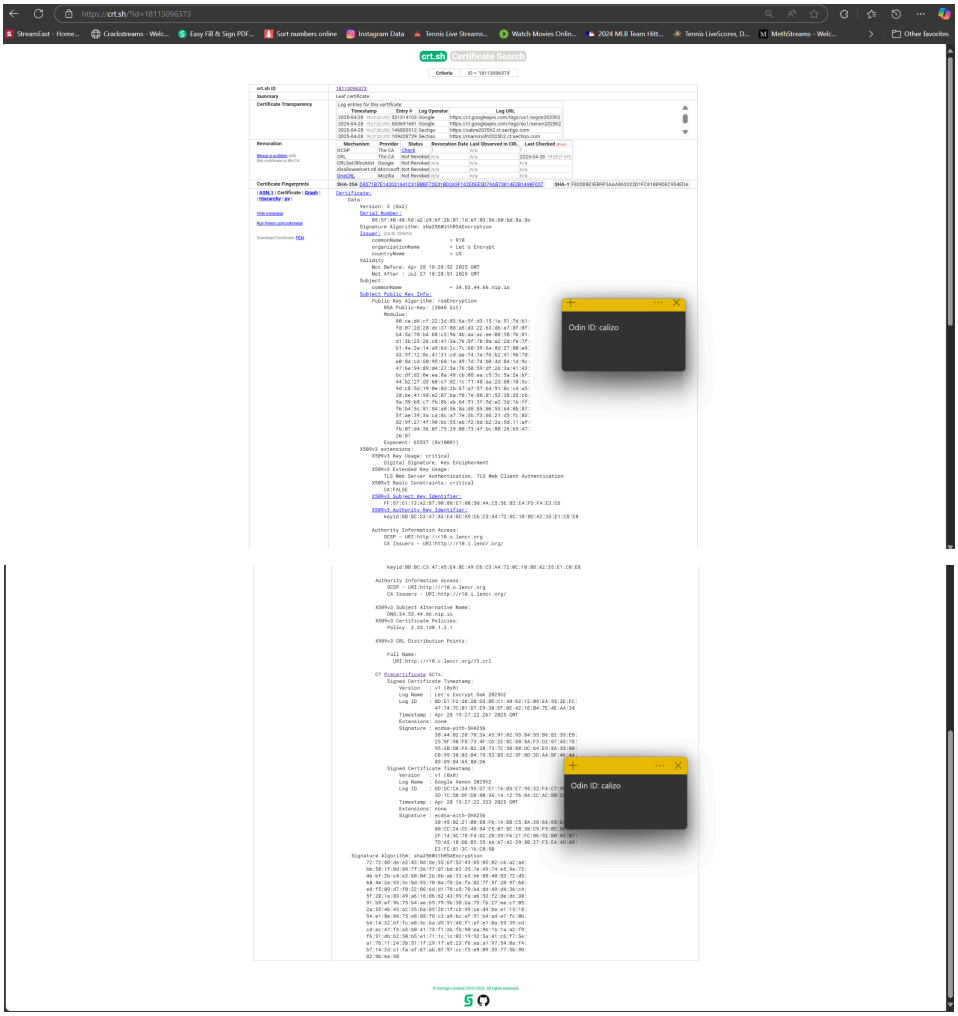
Upon inspection:

- 18113096373 corresponds to the actual final certificate.
- 18113099186 corresponds to the pre-certificate generated for Certificate Transparency logging.

Answer to Question 3:

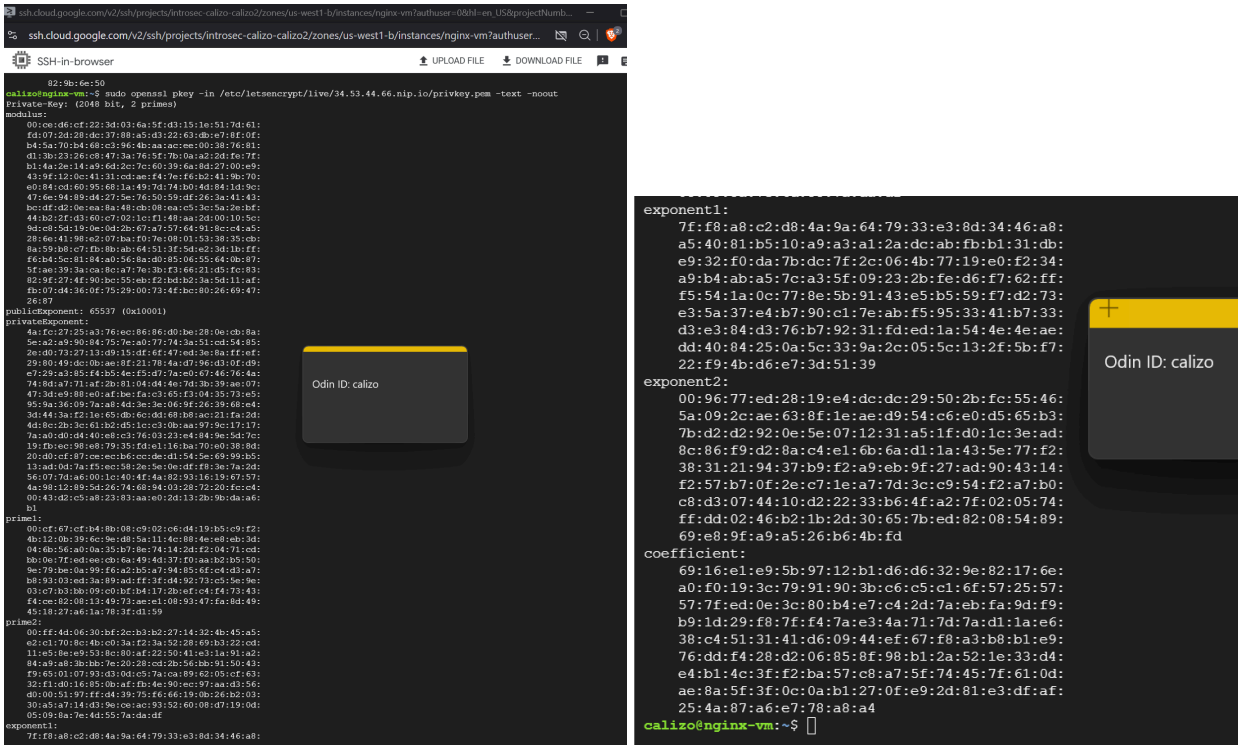
- Therefore, **crt.sh ID 18113096373** is the correct certificate ID to use to match my public site certificate, and a screenshot of it was captured. This certificate matches the public key and signature structure observed earlier.





Finally, to complete the key analysis, I inspected the corresponding private key file (/etc/letsencrypt/live/34.53.44.66.nip.io/privkey.pem) using openssl pkey. I verified the modulus and found the two primes (prime1 and prime2) that were used to construct the RSA private key. A full screenshot was taken showing the two prime numbers along with the private key's other RSA parameters.

Answer to Question 4:
The two primes and the rest of the key information are shown clearly in the screenshots taken from the private key output.



Step 13: Clean Up
After verifying all certificates and saving the required screenshots, I deleted the VM instance to avoid incurring unnecessary Google Cloud charges. I issued the following command from the Cloud Shell:

- gcloud compute instances delete nginx-vm --zone=us-west1-b

This de-provisioned the nginx-vm instance and ensured that my resources were properly cleaned up after completing the lab.